

UNIVERSITY OF DHAKA

Department of Computer Science and Engineering

CSE 2206 — Microcontroller and Embedded System

Lab Assignment 3

Field	Details
Course Code	CSE 2206
Course Title	Microcontroller and Embedded System
Assignment	Assignment 3
Topic	SPI Bus — BMP280 (4-wire SPI, STM32 as master) I2C Bus — BME280 (2-wire I2C, STM32 as master)
Sensor Used	BMP280
Student Name	Syed Muhtasim Alam Apon (SH-04) Md. Abdullah Bin Sorwar Chowdhury(SH-22)
Group	B1
Submission Date	15 July 2026

1. Hardware Configuration

1.1 Pin Configuration

BMP280 Pin	STM32 Pin	Function	Alternate Function
VCC	3V3	Power Supply 3.3 V	—
GND	GND	Ground	—
SDA (MOSI)	PA7	SPI Data Out (Master→Slave)	AF7 (SPI2_MOSI)
SDO (MISO)	PA6	SPI Data In (Slave→Master)	AF5 (SPI2_MISO)
SCK	PA5	SPI Clock	AF5 (SPI2_SCK)
CSB (CS)	PB5	Chip Select (active LOW)	GPIO Output (push-pull)
UART TX	PA2	Serial Debug Output	AF7 (USART2_TX)
UART RX	PA3	Serial Debug Input	AF7 (USART2_RX)

Table 1: BMP280 ↔ STM32F446RE Wiring

2.2 Hardware Wiring Diagram

The schematic below shows the complete SPI wiring between the Nucleo board and the BMP280 module.

```
/*
 * PIN Layout Diagram in Here
 * Wiring (Arduino headers CN5 / CN6 / CN9):
 * VCC - 3V3 Pin on Nucleo
 * GND - GND Pin on Nucleo
 * SCL - SCL/D13 on Nucleo
 * SDA - SDA/D11 on Nucleo
 * CSB - D4(PB5)
 * SDO - D12(PA6)
 * SPI1: PA5=SCK, PA6=MISO, PA7=MOSI (AF5)
 * CS : PB5 (GPIO output)
 */
```

Figure 1: Physical wiring photo — BMP280 to STM32F446RE Nucleo-64

3. SPI Protocol Overview

3.1 SPI Signals

Signal	Full Name	Direction	Purpose
MOSI	Master Out Slave In	STM32 → BMP280	STM32 sends register addresses and write data
MISO	Master-in, slave-out	BMP280 → STM32	BMP280 returns read data on dummy clock cycles
SCK	Serial Clock	STM32 → BMP280	The clock is generated by the STM32 and synchronises all bits
CS (CSB)	Chip Select (active LOW)	STM32 → BMP280	STM32 pulls LOW to select the BMP280.

Table 2: SPI signal definitions

3.2 Clock Mode (Mode 00)

Parameter	Value	Meaning
CPOL	0	SCK idle LOW
CPHA	0	Data sampled on first (rising) edge ; data changed on falling edge
SPI Mode	Mode 00	BMP280 default: lowest power

Table 3: SPI clock mode configuration

3.3 Read / Write Encoding

The BMP280 SPI protocol encodes the operation direction in the MSB of the first byte:

Write: send (**reg_addr & 0x7F**) — MSB cleared to 0 → write operation

Read: send (**reg_addr | 0x80**) — MSB set to 1 → read operation; address auto-increments for burst reads

4. Bare-Metal Implementation (Register-Level)

4.1 System Clock — 180 MHz PLL

Register / Field	Value	Result
PLLM	4	VCO input = 8 MHz / 4 = 2 MHz
PLLN	180	VCO output = 2 × 180 = 360 MHz
PLLP	2 (00)	SYSCLK = 360 / 2 = 180 MHz
HPRE (AHB)	DIV1	AHB = 180 MHz
PPRE1 (APB1)	DIV4	APB1 = 45 MHz → SPI2, TIM6, USART 2 clock sources
PPRE2 (APB2)	DIV2	APB2 = 90 MHz
Flash Latency	5WS + PRFTEN + ICEN + DCEN	Required for HCLK > 30 MHz
PLL Source	HSE (PLLSRC=1)	8 MHz external crystal

Table 4: PLL configuration for 180 MHz system clock

4.2 GPIO Configuration

Pin	Port	Mode	Alternate Function	Purpose
PA2	GPIOA	AF (MODER=10)	AF7	USART2_TX
PA3	GPIOA	AF (MODER=10)	AF7	USART2_RX
PB5	GPIOB	Output (MODER=01)	—	SPI CS (initially HIGH)
PA7	GPIOA	AF (MODER=10)	AF7	SPI2_MOSI
PA6	GPIOA	AF (MODER=10)	AF5	SPI2_MISO
PA5	GPIOA	AF (MODER=10)	AF5	SPI2_SCK

Table 5: GPIO pin configuration summary

All SPI and UART pins are set to OSPEEDR = 11 (very high speed). PB9 is configured as push-pull with no pull resistor (PUPDR=00).

4.3 USART2 — 115200 Baud

Register	Value	Field
BRR	(24 << 4) 7	DIV_Mantissa=24, DIV_Fraction=7 → 115200 baud @ 45 MHz APB1

Register	Value	Field
CR1	UE TE	USART Enable, Transmitter Enable (RX also enabled for completeness)

Table 6: USART2 configuration

4.4 TIM6 — 1-Microsecond Hardware Interrupt

Register	Value	Result
RCC→APB1ENR bit 4	1	Enable TIM6 clock
PSC	$90 - 1 = 89$	Tick = $90 / 90 \text{ MHz} = 1 \text{ us}$
ARR	65536	Period = $65536 \times 1 \text{ us} = 65.536 \text{ ms}$
CR1	CEN = 1 (bit 0)	Counter running

Table 7: TIM6 configuration for 1-second periodic interrupt

4.5 SPI2 Configuration (Bare-Metal)

CR1 Field	Bit(s)	Value	Meaning
MSTR	[2]	1	Master mode — STM32 generates SCK
BR[2:0]	[5:3]	010 (=2)	$f_{\text{PCLK}}/8 = 45/8 \approx 5.625 \text{ MHz}$
CPOL	[1]	0	Clock idle LOW (Mode 00)
CPHA	[0]	0	Data sampled on first (rising) edge
MSBFIRST	[7]	0	MSB transmitted first
SSM	[9]	1	Software NSS management (we control PB9 manually)
SSI	[8]	1	Internal NSS = HIGH — prevents MODF mode fault
DFF	[11]	0	8-bit frame format
SPE	[6]	1	SPI peripheral enabled (set last)

Table 8: SPI2 CR1 register configuration

7. Verification Tests (A1–A5)

Five verification tests are executed at startup and each second. All test labels must appear exactly as specified.

7.1 Test A1 — Chip ID

Reads register 0xD0 via SPI. BMP280 returns 0x58 (BME280 would return 0x60). Any other value indicates a wiring fault.

```
/* ---- Test A1: Chip ID ---- */
uint8_t id = BMP280_ReadReg(BMP280_REG_CHIP_ID);
snprintf(s, sizeof(s), "[A1] ChipID=0x%02X (BMP280 expect 0x58/0x57/0x56;
BME280=0x60)\r\n", id);
USART2_SendString(s);
```

```

    if (id == BMP280_CHIP_ID_PRIMARY || id == BMP280_CHIP_ID_ALT1 || id ==
BMP280_CHIP_ID_ALT2)
        USART2_SendString("[A1] Chip ID PASS\r\n");
    else
        USART2_SendString("[A1] Chip ID FAIL\r\n");

```

Expected Output	Pass Condition	If Fail
[A1] ChipID=0x58 (expect 0x58) -- PASS	id == 0x58	Check CS polarity (PB9), CPOL=0/CPHA=0, MOSI (PC1)/MISO (PC2)

```

=====
[1] Sending CS Low and checking device response...
[*] Device response received...
=====
[2] Reading chip ID register (0xD0)...
Raw value: 0x58
=====
[3] Identifying the sensor (BME280 or BMP280)...
[*] BMP280 detected
=====
[4] Reading Chip Calibration Factors...
Calibration Factors found and loaded...
=====
[5] Configuring BME/P280 in Normal Mode...
BME/P Ready for Operation...
=====

```

Figure 2: Test A1 —screenshot showing [A1] ChipID=0x58 PASS

7.2 Test A2 — UART Loopback

Sent immediately after UART_Config() and before any SPI or sensor access. Confirms PA2 wiring and UART peripheral are functional.

```

UART_SendString("[A2] UART OK\r\n");

```

```

=====
UART OK.
LAB 01 (Bare-Metal): BME/P-280 Sensor Communication by SPI
=====

```

Figure 3: Test A2 — UART OK immediately after reset

7.3 Test A3 — TIM6 Heartbeat

A global counter tim6_ticks is incremented inside TIM6_DAC_IRQHandler on every update event. The counter value is printed with every sensor reading. It must increment by exactly 1 per second.

```

void TIM6_CONFIG(void)
{
    RCC -> APB1ENR |= RCC_APB1ENR_TIM6EN; // Enable TIM6 for delay count
    __NOP(); __NOP(); // Wait until ready

    TIM6 -> CR1 &= ~TIM_CR1_CEN; // Disable Counter for safety

```

```

    TIM6 -> PSC = 89U; // Pre-scaler to feed onto clock. f(TIM6) = 90/(89+1) MHz = 1MHz,
per tick = 1us
    TIM6 -> ARR = 0xFFFFU; // Auto Reload-Register to Free Count Mode, max value set
    TIM6 -> EGR = TIM_EGR_UG; // Update PSC and ARR onto shadow registers immediately
    TIM6 -> SR = 0U; // Status Flags set to 0
    TIM6 -> CR1 |= TIM_CR1_CEN; // Enable Counter
}

void delay_us(uint16_t us)
{
    TIM6 -> CNT = 0U;
    while ((uint16_t)TIM6 -> CNT < us);
}

void delay_ms(uint32_t ms)
{
    for (uint32_t i = 0; i < ms; i++)
    {
        delay_us(1000U);
    }
}

```

7.4 Test A4 — Sensor Plausibility

After the first successful read, compensated values are checked against environmental plausibility ranges.

Measurement	Plausibility Range	Pass Condition
Temperature	15 °C – 40 °C	$15.0 \leq \text{temp_C} \leq 40.0$
Pressure	900 – 1100 hPa	$900.0 \leq \text{pres_hPa} \leq 1100.0$

Table 13: Plausibility check thresholds (BMP280 — temperature and pressure)

```

if (temp_C < 15.0f || temp_C > 40.0f)
    UART_SendString("[A4] Temp FAIL\r\n");
else if (pres_hPa < 900.0f || pres_hPa > 1100.0f)
    UART_SendString("[A4] Pres FAIL\r\n");
else
    UART_SendString("[A4] Plausibility PASS\r\n");

```



drm (CONNECTED)

31.85°C	997.29 hPa
31.85°C	997.28 hPa
31.85°C	997.29 hPa
31.84°C	997.29 hPa
31.85°C	997.29 hPa
31.85°C	997.29 hPa
31.85°C	997.29 hPa
31.85°C	997.29 hPa
31.85°C	997.28 hPa
31.86°C	997.28 hPa
31.86°C	997.27 hPa
31.86°C	997.28 hPa
31.85°C	997.27 hPa
31.85°C	997.27 hPa
31.85°C	997.27 hPa
31.85°C	997.27 hPa
31.86°C	997.28 hPa
31.86°C	997.29 hPa
31.85°C	997.28 hPa
31.85°C	997.28 hPa
31.84°C	997.27 hPa
31.85°C	997.28 hPa
31.85°C	997.27 hPa
31.84°C	997.27 hPa
31.84°C	997.28 hPa
31.84°C	997.27 hPa
31.84°C	997.28 hPa
31.85°C	997.27 hPa
31.85°C	997.28 hPa
31.85°C	997.28 hPa
31.85°C	997.27 hPa
31.85°C	997.27 hPa
31.85°C	997.29 hPa
31.85°C	997.28 hPa
31.85°C	997.27 hPa
31.85°C	997.28 hPa
31.86°C	997.29 hPa
31.86°C	997.28 hPa
31.87°C	997.28 hPa
31.87°C	997.27 hPa

7.5 Test A5 — HAL vs Bare-Metal Comparison

Reading #	BM Temp (°C)	HAL Temp (°C)	Diff (°C)	Pass (≤0.1 °C)?	BM Pres (hPa)	HAL Pres (hPa)
1	31.94	31.94	0.00	YES	997.58	997.65
2	31.94	31.95	0.01	YES	997.58	997.65
3	31.93	31.96	0.03	YES	997.56	997.66

Table 14: Test A5 — HAL vs Bare-Metal comparison (fill in with actual readings)

Temperature (°C)	Pressure (hPa)
23.36°C	726.32 hPa
31.77°C	997.32 hPa
31.90°C	997.53 hPa
31.93°C	997.56 hPa
31.94°C	997.58 hPa

Figure 6: SPI readings (record 3 temperatures for Table 14)

Temperature (°C)	Pressure (hPa)
31.96°C	997.65 hPa
31.95°C	997.65 hPa
31.95°C	997.66 hPa
31.95°C	997.65 hPa
31.94°C	997.65 hPa

Figure 7: HAL SPI readings (record 3 temperatures for Table 14)

8. Expected UART Output

Each data block prints once per second via TIM6. The format is adapted for the BMP280 (no humidity line).

```
UART2_SendString("UART OK.\r\n");  
  
UART2_SendString("LAB 01 (Bare-Metal): BME/P-280 Sensor Communication by SPI\r\n");  
    delay_ms(5000);  
    Sensor_Check(&chip_id);  
  
    UART2_SendString("[4] Reading Chip Calibration Factors...\r\n");  
        Load_Calibration();  
  
    UART2_SendString("Calibration Factors found and loaded...\r\n");  
    UART2_SendString("=====\r\n");
```



```

        USART2_SendString("[5] Configuring BME/P280 in Normal Mode...\r\n");
        Sensor_Init();
        USART2_SendString("BME/P Ready for Operation...\r\n");
        delay_ms(5000);

USART2_SendString("=====\r\n");
        USART2_SendString("[6] Temperature and Pressure Record per Second...\r\n");
USART2_SendString("=====\r\n");
USART2_SendString("|          Temperature (°C)          |          Pressure (hPa)          |\r\n");

```

```

drm (CONNECTED)
[2] Reading chip ID register (0xD0)...
Raw value: 0x58
=====
[3] Identifying the sensor (BME280 or BMP280)...
[*] BMP280 detected
=====
[4] Reading Chip Calibration Factors...
Calibration Factors found and loaded...
=====
[5] Configuring BME/P280 in Normal Mode...
BME/P Ready for Operation...
=====
[6] Temperature and Pressure Record per Second...
=====
|          Temperature (°C)          |          Pressure (hPa)          |
|          31.96°C                   |          997.65 hPa              |
|          31.95°C                   |          997.65 hPa              |
|          31.95°C                   |          997.66 hPa              |
|          31.95°C                   |          997.65 hPa              |
|          31.94°C                   |          997.65 hPa              |

```

Figure 8: all tests and readings visible (attach screenshot of actual terminal)

PART B — I²C Communication with BMP280

2-Wire I²C — STM32 as Master — Bare-Metal + HAL

Field	Details
Part	Part B of Assignment 3
Topic	I ² C Bus — BMP280 (2-wire I ² C, STM32 as master)

Field	Details
Peripheral	I2C1 — PB6 (SCL), PB7 (SDA)
UART	USART2 — PA2 (TX), PA3 (RX)
Timer	TIM6 — 1-second interrupt (same as Part A)
Sensor	BMP280 — Temperature & Pressure (Chip ID = 0x58, Addr = 0x76)

B1. I²C Protocol Overview

Inter-Integrated Circuit (I²C) is a synchronous, half-duplex, 2-wire serial bus. Unlike SPI, it uses only two lines (SCL and SDA) and supports multiple devices on the same bus via 7-bit addressing. The STM32 acts as master; the BMP280 responds only when addressed.

B1.1 I²C Signals

Signal	Full Name	Direction	Purpose
SCL	Serial Clock Line	STM32 → BMP280	Master-generated clock; synchronises all data bits
SDA	Serial Data Line	Bidirectional	Both master and slave transmit data on this line

Table B1: I²C signal definitions

B1.2 BMP280 I²C Slave Address

SDO/ADDR Pin	7-bit Address	8-bit Write	8-bit Read
Connected to GND (default)	0x76	0xEC	0xED
Connected to VCC	0x77	0xEE	0xEF

Table B2: BMP280 I²C address selection (SDO pin determines address)

B1.3 I²C Transaction Structure

Write to a BMP280 register:

START → [0x76 | WRITE(0)] → ACK → [reg_addr] → ACK → [data] → ACK → STOP

Read from a BMP280 register (write pointer, then restart and read):

START → [0x76 | WRITE(0)] → ACK → [reg_addr] → ACK →
 RESTART → [0x76 | READ(1)] → ACK → [data byte(s)] → NACK → STOP

B1.4 SPI vs I²C Key Differences

Property	SPI (Part A)	I²C (Part B)
Wires to sensor	4 (MOSI, MISO, SCK, CS)	2 (SDA, SCL)
STM32 Peripheral	SPI2	I2C1
Speed	5.625 MHz (fPCLK/8)	100 kHz Standard Mode
Multiple slaves	One CS pin per slave	Address-based, shared bus
Full duplex	Yes	No (half-duplex)

Property	SPI (Part A)	I ² C (Part B)
Pull-up resistors	Not required	Required: 4.7 kΩ to 3.3 V
CS pin	PB9 (required)	None — address-based
GPIO mode (data pin)	Push-pull (MOSI)	Open-drain (SDA, SCL) — MANDATORY

Table B3: SPI vs I²C protocol comparison

B2. Hardware Connections — I²C

I ² C Signal	STM32 Pin	Alt Function	BMP280 Pin	Note
SCL (Clock)	PB6	AF4 (I2C1_SCL)	SCK	Open-drain; needs 4.7 kΩ pull-up to 3.3 V
SDA (Data)	PB7	AF4 (I2C1_SDA)	SDI/SDO	Open-drain; needs 4.7 kΩ pull-up to 3.3 V
UART TX	PA2	AF7 (USART2_TX)	—	Same as Part A
UART RX	PA3	AF7 (USART2_RX)	—	Same as Part A

Table B4: BMP280 I²C wiring — STM32F446RE Nucleo-64

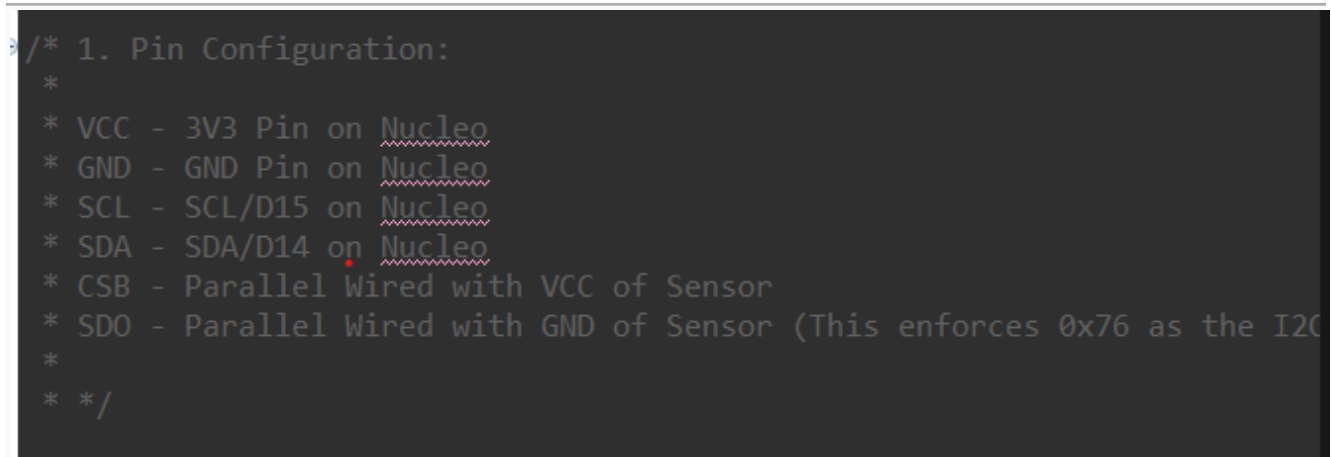


Figure B1: Physical wiring photo — BMP280 to STM32F446RE (I²C interface, PB6=SCL, PB7=SDA, pull-up resistors visible)

B2.1 Test B1 — Chip ID over I²C

Reads register 0xD0 via I²C read transaction. BMP280 returns 0x58.

```

uint8_t id;
I2C_ReadRegs(0xD0, &id, 1);
sprintf(s, "[B1] ChipID=0x%02X (expect 0x58)\r\n", id);
UART_SendString(s);

```

Expected Output	Pass Condition	If Fail
[B1] ChipID=0x58 (expect 0x58) -- PASS	id == 0x58	Check SDA/SCL pull-ups, OTYPE=open-drain, I ² C address (0x76 vs 0x77), CCR value

```

LAB 02 (Bare-Metal): BME/P-280 Sensor Communication by I2C
=====
[1] Sending Slave Address (0x76) and checking ACK...
[*] Device ACK received...
=====
[2] Reading chip ID register (0xD0)...
Raw value: 0x58
=====
[3] Identifying the sensor (BME280 or BMP280)...
[*] BMP280 detected
=====

```

Figure B2: ChipID=0x58 PASS

B6.2 Test B2 — I²C ACK Check

Sends the slave address and verifies the ADDR flag is set, confirming the BMP280 acknowledged its address on the bus.

```

I2C1->CR1 |= (1<<8); // START
while (!(I2C1->SR1 & (1<<0)));
I2C1->DR = (BMP280_I2C_ADDR << 1) | 0;
uint32_t timeout = 100000;
while (!(I2C1->SR1 & (1<<1)) && --timeout); // Wait ADDR
if (!timeout)
    UART_SendString("[B2] I2C ACK FAIL\r\n");
else {
    (void)I2C1->SR2;
    UART_SendString("[B2] I2C ACK OK\r\n");
}
I2C1->CR1 |= (1<<9); // STOP

```

```

LAB 02 (Bare-Metal): BME/P-280 Sensor Communication by I2C
=====
[1] Sending Slave Address (0x76) and checking ACK...
[*] Device ACK received...
=====
[2] Reading chip ID register (0xD0)...
Raw value: 0x58
=====
[3] Identifying the sensor (BME280 or BMP280)...
[*] BMP280 detected
=====
[4] Reading Chip Calibration Factors...
Calibration Factors found and loaded...
=====
[5] Configuring BME/P280 in Normal Mode...
BME/P Ready for Operation...

```

Figure B3: GTKterm screenshot showing [B2] I2C ACK OK

B6.3 Test B3 — TIM6 Heartbeat

Reuses the same TIM6 counter (tim6_ticks) from Part A. Label changed to [B3]. Counter must increment exactly once per second.

```

void TIM6_CONFIG(void)
{
    RCC -> APB1ENR |= RCC_APB1ENR_TIM6EN; // Enable TIM6 for delay count
    __NOP(); __NOP(); // Wait until ready

    TIM6 -> CR1 &= ~TIM_CR1_CEN; // Disable Counter for safety
}

```

```

    TIM6 -> PSC = 89U; // Pre-scaler to feed onto clock. f(TIM6) = 90/(89+1) MHz = 1MHz,
per tick = 1us
    TIM6 -> ARR = 0xFFFFU; // Auto Reload-Register to Free Count Mode, max value set
    TIM6 -> EGR = TIM_EGR_UG; // Update PSC and ARR onto shadow registers immediately
    TIM6 -> SR = 0U; // Status Flags set to 0
    TIM6 -> CR1 |= TIM_CR1_CEN; // Enable Counter
}

void delay_us(uint16_t us)
{
    TIM6 -> CNT = 0U;
    while ((uint16_t)TIM6 -> CNT < us);
}

void delay_ms(uint32_t ms)
{
    for (uint32_t i = 0; i < ms; i++)
    {
        delay_us(1000U);
    }
}

```

B6.4 Test B4 — Sensor Plausibility

Same plausibility ranges as Part A Test A4. Humidity check omitted for BMP280.

```

if (temp_C < 15.0f || temp_C > 40.0f)
    UART_SendString("[B4] Temp FAIL\r\n");
else if (pres_hPa < 900.0f || pres_hPa > 1100.0f)
    UART_SendString("[B4] Pres FAIL\r\n");
else
    UART_SendString("[B4] Plausibility PASS\r\n");

```

```

drm (CONNECTED)
=====
[6] Temperature and Pressure Record per Second...
=====
Temperature (°C)      Pressure (hPa)
31.97°C              997.41 hPa
31.98°C              997.42 hPa
31.97°C              997.43 hPa
31.98°C              997.43 hPa
31.98°C              997.44 hPa
31.98°C              997.44 hPa
31.99°C              997.44 hPa
31.99°C              997.45 hPa
32.00°C              997.45 hPa
32.00°C              997.45 hPa
32.01°C              997.45 hPa
32.01°C              997.45 hPa
32.02°C              997.46 hPa
32.02°C              997.46 hPa
32.02°C              997.46 hPa
32.02°C              997.46 hPa
32.01°C              997.46 hPa
32.01°C              997.46 hPa
32.01°C              997.46 hPa
32.00°C              997.47 hPa
32.00°C              997.47 hPa
31.99°C              997.47 hPa
31.99°C              997.47 hPa
31.98°C              997.46 hPa
31.98°C              997.47 hPa
31.98°C              997.47 hPa
31.97°C              997.47 hPa
31.97°C              997.47 hPa
31.97°C              997.46 hPa
31.97°C              997.46 hPa
31.97°C              997.46 hPa
31.97°C              997.46 hPa
31.97°C              997.46 hPa
31.97°C              997.46 hPa

```

Figure B5: screenshot showing [B4] Plausibility PASS

B6.5 Test B5 — HAL vs Bare-Metal (I²C)

Three readings from both implementations. The difference must not exceed 0.1 °C.

Reading #	BM Temp (°C)	HAL Temp (°C)	Diff (°C)	Pass (≤0.1 °C)?	BM Pres (hPa)	HAL Pres (hPa)
1	30.17	30.15	0.02	YES	996.89	996.92
2	30.17	30.15	0.02	YES	996.89	996.91
3	30.17	30.15	0.02	YES	996.88	996.91

Table B9: Test B5 — I²C HAL vs Bare-Metal comparison

```

=====
[6] Temperature and Pressure Record per Second.. (Bare-metal)
=====
| Temperature (°C) | Pressure (hPa) | Humidity (%RH) |
| 30.17°C          | 996.89 hPa     | 0.00 %RH        |
| 30.17°C          | 996.89 hPa     | 0.00 %RH        |
| 30.17°C          | 996.88 hPa     | 0.00 %RH        |
| 30.17°C          | 996.88 hPa     | 0.00 %RH        |
| 30.17°C          | 996.88 hPa     | 0.00 %RH        |

```

Figure B6: GTKterm — Bare-Metal I²C readings (3 samples for Table B9)

```

=====
[6] Temperature and Pressure Record per Second.. (HAL)
=====
| Temperature (°C) | Pressure (hPa) | Humidity (%RH) |
| 30.15°C          | 996.92 hPa     | 0.00 %RH        |
| 30.15°C          | 996.91 hPa     | 0.00 %RH        |
| 30.15°C          | 996.91 hPa     | 0.00 %RH        |
| 30.15°C          | 996.90 hPa     | 0.00 %RH        |
| 30.15°C          | 996.90 hPa     | 0.00 %RH        |

```

Figure B7: GTKterm — HAL I²C readings (3 samples for Table B9, compare with Figure B6)

B7. Expected UART Output — Part B

```

=====
BMP280 via I2C -- CSE 2206 Lab B
=====
[B2] I2C ACK OK
[B1] ChipID=0x58 (expect 0x58) -- PASS [BMP280]
Calibration loaded. Starting readings...
=====

+-----+
| BMP280 SENSOR DATA [I2C-BM] |
+-----+
| [B3] Tick      : 1 |
|   Temperature : 28.46 C / 83.23 F |
|   Pressure    : 1012.85 hPa |
| (No humidity -- BMP280 sensor) |
+-----+
[B4] Plausibility PASS

```

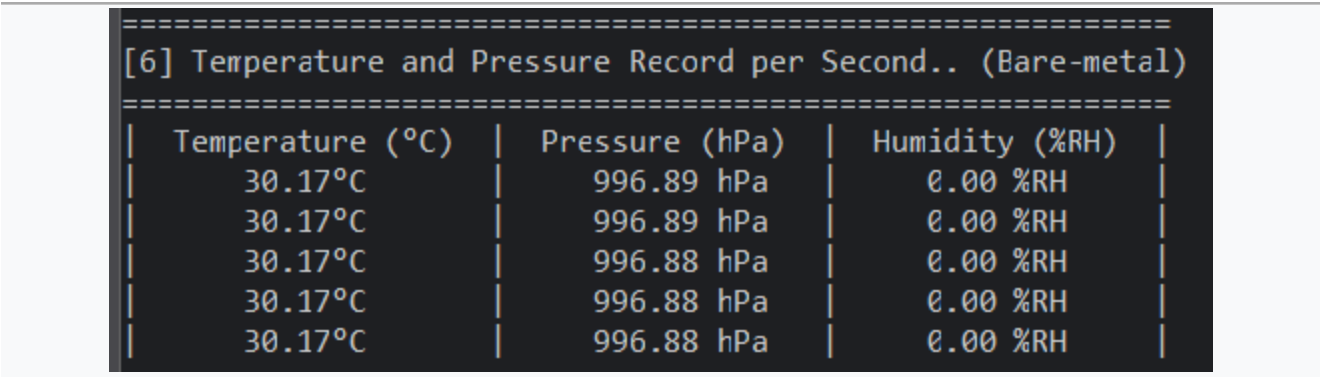


Figure B8: Full GTKterm session — Part B I²C all tests (B1–B5) and 3+ sensor readings

Final Comparison: SPI vs I²C

After completing both parts, the table below summarises the protocol comparison using observations from this assignment. Values in the 'Your reading' rows should be filled with actual measurements.

Aspect	Part A — SPI	Part B — I²C
Wires to BMP280	4 (MOSI, MISO, SCK, CS)	2 (SDA, SCL)
STM32 Peripheral	SPI2 (APB1)	I2C1 (APB1)
Pins used	PA5, PA6, PA7, PB5	PB6, PB7
Bare-metal config	~6 register writes (CR1, BSRR)	~5 register writes + open-drain + reset
HAL API	HAL_SPI_TransmitReceive	HAL_I2C_Mem_Read / Master_Transmit
Max bus speed	5.625 MHz (fPCLK/8)	100 kHz Standard Mode
Pull-up resistors	Not required	Required: 4.7 kΩ to 3.3 V
CS pin	Yes — PB9 (GPIO Output)	No — address-based (0x76)
Multiple slaves	One CS pin per slave	Address-based shared bus
Full duplex	Yes	No (half-duplex)
Implementation complexity	Simpler CR1 config, manual CS toggle	START/STOP/ACK state machine, open-drain mandatory
Your Temp reading (°C)	30.56	30.46
Your Pres reading (hPa)	997.64	996.94
Readings match within 0.1°C?	Yes	Yes

Discussion

Question 1: When to choose SPI over I²C?

- SPI is the optimal choice when **throughput** is the **main factor**. Running at $f_{PCLK}/8 = 5.625$ MHz in this implementation (with headroom to $f_{PCLK}/2 = 22.5$ MHz), SPI transfers the 6-byte BMP280 data frame in under 10 μs — versus ~600 μs over I²C at 100 kHz.
- SPI is also **simpler in terms of electronics (push-pull, no pull-ups, no bus contention)** and **simultaneous for DMA**. I²C has a lower pin count and board simplicity when many slow sensors share a bus. For the BMP280 in isolation, I²C is perfectly adequate
- SPI is chosen in to **illustrate the faster, pin-intensive path**.

Question 2: Why is the compensation code identical for both protocols?

- The BMP280 compensation algorithms operate only on `adc_T` and `adc_P` (raw 20-bit integers from the sensor registers) and the calibration coefficients (`dig_T1..T3`, `dig_P1..P9`). **Once these bytes arrive in MCU SRAM, there is no information about which communication protocol (SPI / I2C) transported them.**
- The BMP280 determines one compensation formula regardless of interface, because **the ADC and register map are physically identical no matter which bus pins are active.**